# Adjacency Rows vs. Laplacian Eigenvectors

Why the "weaker" encoding won the isomorphism task — and what each one really tells the model

isomorphism · `src.dataset.tokenize_dataset` · `src.features.laplacian_positional_encoding` · 2026-06-23

**The puzzle.** Adjacency-row tokenization tops out at ~0.85 test accuracy. Laplacian-eigenvector tokenization — the "real" structural encoding — never clears 0.60. The intuition that structural encoding > raw adjacency was reasonable, but it is **backwards for this dataset**. The reason is not that Laplacian encodings are weak in general; it is that **this task is secretly a degree-sequence comparison**, and the two encodings sit on opposite sides of that fact: adjacency rows hand degree to the model for free, while the normalized Laplacian deliberately throws degree away.

## 1 The task is not the task you think it is

Each sample is a **pair** $(G_1, G_2)$ packed into one disconnected graph: $G_1$ on nodes $0...n-1$, $G_2$ on nodes $n...2n-1$. The label is whether they are isomorphic. The architecture (one global-attention layer, then `pair` pooling that averages $G_1$ nodes and $G_2$ nodes separately and concatenates $\left[h_{G_1} \mid h_{G_2}\right]$) means the classifier's whole job is: **compare a summary of $G_1$ against a summary of $G_2$.**

Now look at how the negatives are made (`make_isomorphism_dataset`, `src/dataset.py`):

- **label 1 (iso):** $G_2$ is a random **permutation** of $G_1$. Same degree sequence by construction.
- **label 0 (non-iso):** $G_2$ is regenerated until its degree sequence **differs** from $G_1$ (the generator loops up to 200 times to force this).

Measured on the actual cached 1000-pair dataset:

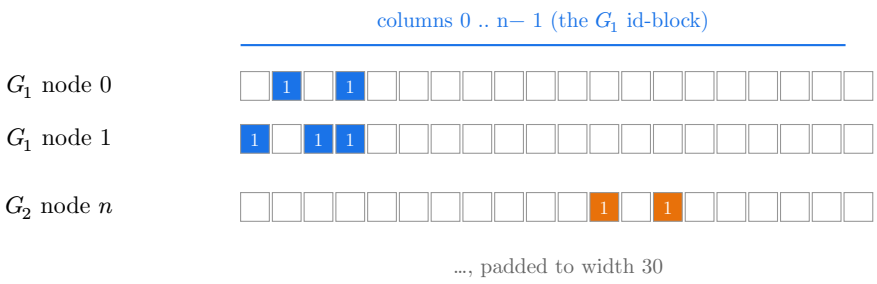
| Class | Pairs whose two components share a degree sequence |
|---|---|
| iso (500) | 500 / 500 |
| non-iso (500) | 0 / 500 |

So the degree sequence is a **perfect** separator. A model that does nothing but "compute each component's degree multiset and check whether they match" would score 100%. The task never asks the model to solve the genuinely hard part of isomorphism (telling apart non-isomorphic graphs that **share** a degree sequence — cospectral mates, regular graphs, etc.). It only ever asks: **do the degree sequences agree?**

This reframes everything. The winning encoding is whichever one makes **degree** easiest to read and compare.

## 2 What an adjacency-row token looks like

Each node becomes its row of the adjacency matrix, zero-padded to the dataset-wide maximum width (here $\max 2n = 30$, so each token is a 30-dim binary vector). Node $i$ has a 1 in column $j$ iff $i \sim j$.




Two facts about this token decide the whole story.

**It leaks degree, for free.** The number of 1s in a row *is* that node's degree. A single linear layer can read $\sum_j x_{ij} = \deg(i)$. `pair` pooling then averages rows within a component; column $j$ of the pooled $G_1$ vector becomes $\deg(j)/n$. The classifier receives, in effect, both components' degree profiles side by side — exactly the separating signal.

**It hides the correspondence between the two graphs.** $G_1$'s 1s live in columns $0 \ldots n-1$; $G_2$'s live in columns $n \ldots 2n-1$. The two components occupy **disjoint column blocks**, so the raw vectors are never directly comparable — a node and its image under the isomorphism have orthogonal token vectors. The model cannot match node-to-node; it can only compare **permutation-invariant summaries** of each block. Fortunately degree is exactly such a summary, and it is all the task needs.

That tension is the ceiling. The model reliably extracts and compares degree distributions ( easy, leaked ) but cannot recover the true vertex correspondence ( hidden ). It therefore lands near the accuracy of a degree-sequence test — about 0.85, missing only the non-iso pairs whose degree sequences differ by a hair (`analyze_iso.py` confirms the errors concentrate at small `deg_seq_diff`).

# 3 What a Laplacian-eigenvector token looks like

Each node becomes its row of the matrix of the $k = 16$ smallest non-trivial eigenvectors of the **symmetric normalized** Laplacian $L = I - D^{-1/2} A D^{-1/2}$. The promise: a coordinate system that depends only on graph shape, so isomorphic graphs get matching encodings. Three things break that promise **on this particular dataset**.

## 3.1 Break 1 — normalization erases degree

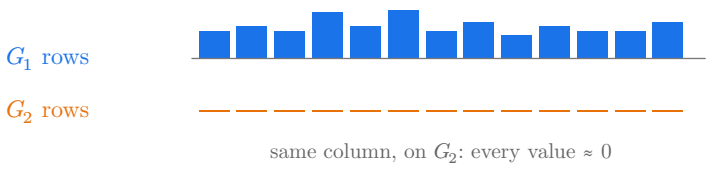
The $D^{-1/2}$ factors are there precisely to make the Laplacian scale-free: they divide out degree. Empirically, on a real pair from the dataset, the correlation between a node's eigenvector-row norm and its degree is

$$\text{corr}(\text{LPE row norm}, \deg) = 0.086 \approx 0.$$

The one feature that perfectly solves the task is the one feature the normalized Laplacian is **designed** to remove. The encoding spends its 16 dimensions describing **relative position** — a question the task never asks — and says almost nothing about **degree** — the only question it does ask.

## 3.2 Break 2 — the pair is disconnected, so the low eigenvectors are arbitrary

The number of zero eigenvalues of $L$ equals the number of connected components. The packed pair $G_1 \sqcup G_2$ has (at least) **two** components, so eigenvalue 0 has multiplicity $\geq 2$. `laplacian_positional_encoding` skips only eigenvector 0 and keeps eigenvector 1 as its first column — but eigenvector 1 lives **inside** that degenerate null space, where `eigh` returns an **arbitrary orthonormal basis**. On a real iso pair from the cache, the kept "first eigenvector" came out as:



The vector is nonzero on $G_1$ and **exactly zero on $G_2$** : `eigh` happened to align the 2-D null space so one basis vector sits entirely on $G_1$, the other entirely on $G_2$. So the model's "first structural coordinate" is really a **component indicator**, carrying zero information about the other graph — and which graph it lands on is an arbitrary numerical accident that flips from pair to pair.

### 3.3 Break 3 — isomorphic components are cospectral, so every eigenspace is degenerate

When $G_1$ and $G_2$ are isomorphic they have identical spectra, so the union's eigenvalues come in **exact duplicate pairs**. On the same sample:

$$\text{smallest six eigenvalues} = (0.000,\ 0.000, 0.360,\ 0.360, 0.515,\ 0.515, \ldots)$$

Every eigenvalue has multiplicity 2. That means the **entire** eigenbasis lives in 2-D degenerate subspaces, each spanned by one $G_1$-mode and one $G_2$-mode, and `eigh` hands back an **arbitrary rotation** within each. So even setting aside the usual sign ambiguity ($\pm$ per eigenvector), the eigenvector rows of $G_1$ and $G_2$ are related by an unknown, per-eigenspace rotation that **mixes the two components together**. The "approximately permutation-equivariant" hope in the config comment collapses: there is no fixed transform the classifier can learn to undo.

## 4 Side by side: what each leaks, what each hides

| | **Adjacency rows** (in=30) | **Laplacian eigvecs** (in=16) |
|---|---|---|
| Degree (the separating signal) | Leaked — row sum = degree | Erased — normalization divides it out (corr 0.09) |
| Cross-graph node matching | Hidden — $G_1/G_2$ in disjoint column blocks | Hidden — sign + rotation ambiguity per eigenspace |
| Stability of the token | Deterministic given node ids | Arbitrary basis in degenerate / null spaces |
| Effect of disconnected pair | None — rows are local | Severe — multiplicity-2 null space → indicator vectors |
| Effect of iso pair being cospectral | None | Severe — every eigenvalue doubled → full mixing |
| What it spends capacity describing | Local neighbourhoods (incl. degree) | Relative position — *which the task never asks about* |
| Result | ~0.85 (degree-test ceiling) | ~0.60 (barely above chance) |

## 5 So was the intuition wrong?

Not in general — it was **mismatched to the benchmark**. Laplacian eigenvectors are the stronger encoding for tasks that genuinely require global structure and where the discriminating graphs **share** coarse statistics like degree. This benchmark is the opposite: its negatives are pre-separated by degree, and it packs two graphs into one disconnected, cospectral blob — the single worst input for Laplacian PE, because it maximizes eigenspace degeneracy. Adjacency rows win not by being more expressive but by **leaking the one statistic that happens to be a perfect label**.

**Two ways to make the comparison fair — and actually test the original hypothesis.**

1. **Compute LPE per component, not on the union.** Run the eigendecomposition on $G_1$ and $G_2$ separately (no shared null space, no cross-component rotation), and add sign-invariant

post-processing (e.g. SignNet, or feed $|v|$ / $vv^\top$). This removes Breaks 2–3, leaving a real test of whether **position** helps.

2. **Make the task need structure, not degree.** Generate non-iso negatives that are **degree-preserving** (degree-sequence-matched, or even cospectral). Then adjacency-row degree-reading collapses to chance, and a structural encoding has something to prove. This is the experiment that would support the thesis claim.